

classification_cma_es

Як працює програма — гайд для Богдана

Практична частина дипломної роботи: порівняння класифікаторів із застосуванням розширеного алгоритму CMA-ES (за роботою Літвінчук Ю.А., 2024) та узагальнених адитивних моделей (GAM) з розділу 1 диплома.

У цьому документі — пояснення кожної частини програми, схематичні діаграми та інструкції з запуску.

Зміст

1. Що це за програма
2. Як побудовано — архітектура
3. Дані
4. П'ять базових класифікаторів
5. CMA-ES — серце роботи
6. CMA-NN — п'ятий метод
7. Підбір гіперпараметрів
8. Як вимірюємо якість
9. MLflow — журнал експериментів
10. Як запустити
11. Як читати результати у дашборді
12. FAQ для захисту

1. Що це за програма

Це програмна частина твоєї дипломної роботи. Вона бере справжні набори даних, прокручує їх через декілька класифікаторів, вимірює якість прогнозів і порівнює моделі між собою.

Мета

- Реалізувати п'ять класифікаторів — традиційні (LogReg, SVM, kNN, MLP) плюс окремо метод, описаний у роботі Літвінчук Ю.А. (CMA-NN).
- Додати GAM-класифікатор — це прямий зв'язок із розділом 1 твого диплома про узагальнені адитивні моделі.
- Прогнати їх через стандартну методику (train/test + k-fold) на трьох сучасних наборах даних.
- Виміряти три метрики: Accuracy, F1-score, ROC-AUC.
- Журналити кожен запуск у MLflow для відтворюваності.

Як це пов'язано з дипломом

У дипломі два теоретичні розділи: розділ 1 про GAM (узагальнені адитивні моделі) та розділ 2 про CMA-ES (еволюційна стратегія з адаптацією коваріаційної матриці). Програма ОБИДВА теоретичні розділи перетворює у працюючий код:

- GAM → файл `classifiers/gam.py`, модель ``gam`` у списку.
- CMA-ES → класичний у `classifiers/cma_es.py`, розширений зі сумішами (за дисертацією Літвінчук) у `classifiers/mixture_cma_es.py`.
- Спеціальна модель ``tuned_gam`` — це коли CMA-ES (розділ 2) підбирає оптимальні параметри GAM (розділ 1). Це і є місток між обома теоретичними частинами.

2. Як побудовано — архітектура

Програма організована як Python-пакет `classifiers`, де кожен модуль відповідає за одне завдання. Зверху — точки входу (CLI-скрипт і Streamlit-дашборд), знизу — pipeline, який оркеструє все разом.



Рис. 1. Блочна схема модулів і потоків даних.

Як читати схему

- Помаранчевий шар — підготовка даних (завантажити CSV, нормалізувати ознаки, розбити на train/test/CV).
- Зелений шар — самі моделі (sklearn-сумісні класифікатори).
- Темно-синій шар — те, що використовує CMA-ES (5-й метод і tuning).
- Жовтий шар — допоміжне (метрики, MLflow, оркестратор).
- Фіолетовий — як ти запускаєш все це (термінал або браузер).

Файлово це виглядає так:

```

classifiers/      # сам пакет
data.py           # завантаження датасетів
preprocessing.py  # scale / one-hot / impute
crossval.py       # train/test + k-fold
metrics.py        # accuracy / F1 / AUC
models.py         # LogReg, SVM, kNN, MLP
  
```

```
gam.py          # GAM (розділ 1 диплома)
cma_es.py       # класичний CMA-ES (розділ 2)
mixture_cma_es.py # розширений CMA-ES + EM (Літвінчук)
cma_nn.py       # CMA-NN — 5-й метод
hyperparam_tuning.py # tuned_*
pipeline.py     # оркестратор
tracking.py     # MLflow
scripts/run_experiment.py # CLI
app.py         # Streamlit GUI
tests/        # pytest
```

3. Дані

Програма працює з трьома датасетами. Якщо є kaggle.json — береться Kaggle-варіант (2024). Якщо немає — автоматично завантажується UCI-аналог. Куратор просив "хоч би 1 із 3 був ≤ 2 -3 років" — ця умова закрита датасетом PhiUSIIL (2024).

PhiUSIIL Phishing URL (UCI #967, 2024)

- 235 795 URL-адрес із 54 числовими ознаками (довжина, кількість крапок, наявність HTTPS і т.д.).
- Бінарна класифікація: 1 = справжній сайт, 0 = phishing.
- Завантажується автоматично через бібліотеку ucimlrepo.

Steel Plate Defects (UCI #198 / Kaggle PS S4E3)

- 1 941 (UCI) або 19 219 (Kaggle) металевих пластин з 27 ознаками.
- Мультикласова задача — 7 типів дефектів: Pastry, Z_Scratch, K_Scratch, Stains, Dirtiness, Bumps, Other_Faults.
- За замовчуванням — UCI fallback (без Kaggle).

Default of Credit Card Clients (UCI #350 / Kaggle PS S4E10)

- 30 000 (UCI) або 58 645 (Kaggle) клієнтів банку.
- Бінарна задача: чи буде клієнт у дефолті в наступному місяці.
- Класи незбалансовані (~78% не-дефолт / 22% дефолт) — це робить задачу складнішою і метрики цікавішими.

Препроцесинг (preprocessing.py)

Незалежно від датасета, перед навчанням завжди робиться:

- Імпутація — пропущені значення заповнюються медіаною (числові) або найчастішим значенням (категоріальні).
- StandardScaler — числові ознаки приводяться до середнього 0 і відхилення 1 (важливо для SVM, kNN, NN).
- OneHotEncoder — категоріальні ознаки розгортаються у бінарні колонки.

4. П'ять базових класифікаторів

Куратор просив 5 моделей: LogReg, SVM, NN, kNN і метод із роботи Літвінчук Ю.А. Плюс додали GAM, бо його теорія розписана у розділі 1 диплома — без нього розділ 1 був би відірваний від практичної частини.

LogisticRegression (logreg)

Найпростіша лінійна модель. Шукає коефіцієнти $w_1, w_2, \dots$ так, щоб логіт $\sigma(w \cdot x)$ добре розділяв класи. Швидка і інтерпретована.

SVM (svm)

Метод опорних векторів з ядром RBF. Знаходить розділяюче багатовимірне 'тіло', максимально віддалене від точок обох класів. Гнучкий, але повільний на великих даних.

kNN (knn)

k-найближчих сусідів. Не "вчиться" взагалі — просто запам'ятовує тренувальні дані. Для нової точки бере k найближчих і голосує.

MLP (mlp)

Багатошарова нейронна мережа (sklearn). За замовчуванням — два приховані шари $64 \rightarrow 32$ нейрони. Навчається методом зворотного поширення (gradient descent).

GAM (gam) — модель з розділу 1 диплома

GAM: кожна ознака має свою гладку функцію, потім всі додаються

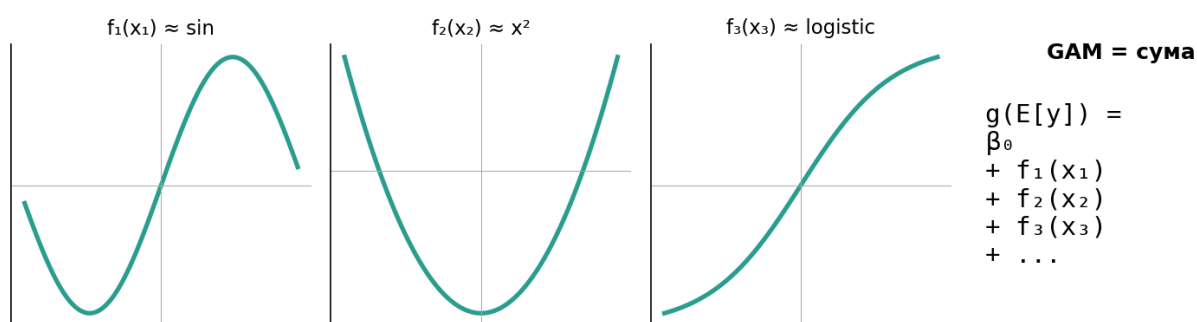


Рис. 2. GAM = сума гладких функцій від кожної ознаки.

GAM (Generalized Additive Model) — це коли кожену ознаку x_i пропускають через свою гладку функцію f_i (зазвичай — сплайн), а потім всі $f_i(x_i)$ додаються. У нашій реалізації функції — це B-сплайни, а підсумовування виконує LogisticRegression.

Параметри GAM (з розділу 1 диплома)

- `n_knots` — кількість вузлів сплайну (контроль гнучкості).
- `degree` — степінь сплайну (3 = кубічний за замовчуванням).
- $C = 1/\lambda$ — обернений параметр згладжування (штраф за надмірну складність функції, розд. 1.5).

5. CMA-ES — серце роботи

CMA-ES (Covariance Matrix Adaptation Evolution Strategy) — це оптимізатор без похідних. Він шукає мінімум функції, не обчислюючи її градієнт, а тільки значення в декількох точках. Це основа того, що описано у розділі 2 диплома.

Класичний CMA-ES (cma_es.py)



Рис. 3. Цикл класичного CMA-ES.

На кожній ітерації:

- 1. Семплуємо N точок x_i з нормального розподілу $N(\mu, \sigma^2 C)$. μ — центр пошуку, σ — масштаб, C — коваріаційна матриця.
- 2. Обчислюємо цільову функцію $f(x_i)$.
- 3. Беремо μ найкращих і використовуємо їх для оновлення.
- 4. Оновлюємо середнє μ (зсувається в бік кращих), коваріацію C ("витягається" у напрямку успіху) і масштаб σ .

У нас класичний CMA-ES — це тонка обгортка над пакетом `cma` (той самий, що використовує Літвінчук у дисертації).

Розширений CMA-ES зі сумішами (mixture_cma_es.py)

Основна ідея Літвінчук Ю.А.: замінити один нормальний розподіл на суміш k нормальних. Тоді алгоритм може досліджувати кілька регіонів простору одночасно — корисно на функціях з багатьма локальними мінімумами.

Розширений CMA-ES зі сумішами (за Літвінчук Ю.А.)

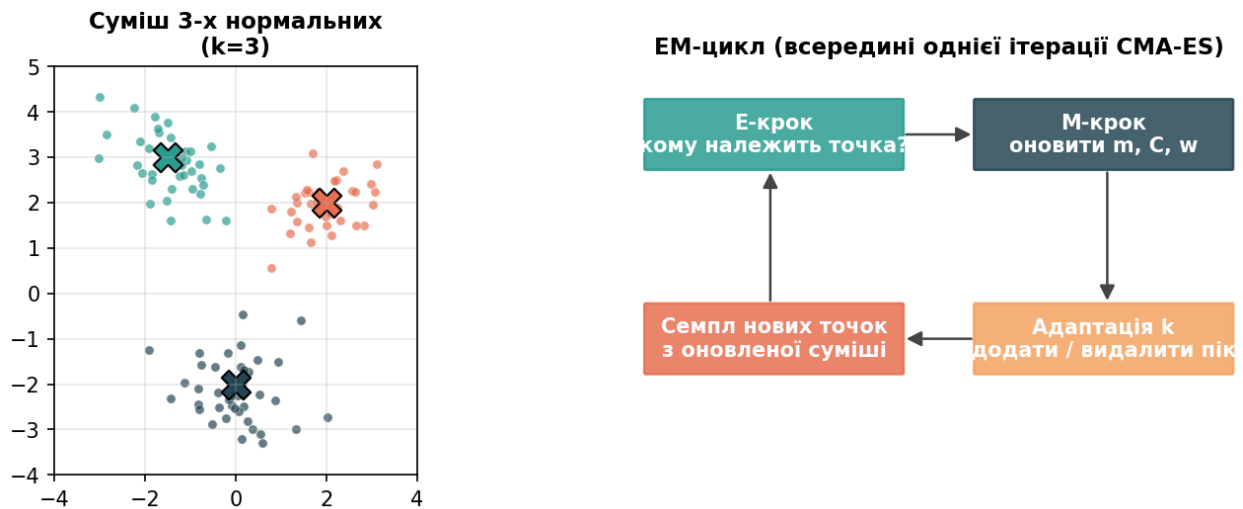


Рис. 4. Суміш 3-х нормальних і цикл ЕМ всередині.

Параметри суміші — ваги w_s , центри m_s , коваріації C_s — оцінюються ЕМ-алгоритмом по найкращих точках.

Що я додав від себе

Чисте ЕМ-оновлення (як описано у дисертації) на унімодальних задачах призводить до передчасного колапсу дисперсії — алгоритм застряє. Я додав параметр `cov_lr` (за замовчуванням 0.2), який змішує нову коваріацію зі старою (як rank-μ оновлення у класичному CMA-ES). Це фікс, якого у дисертації немає.

6. CMA-NN — п'ятий метод

Це і є той самий "метод із роботи Літвінчук", що його просив куратор. Ідея проста: всі ваги невеликої нейромережі укладаємо в один плоский вектор w і шукаємо такий w , який мінімізує cross-entropy на тренувальних даних — за допомогою CMA-ES (класичного або розширеного зі сумішами).

CMA-NN: неймережа очима CMA-ES

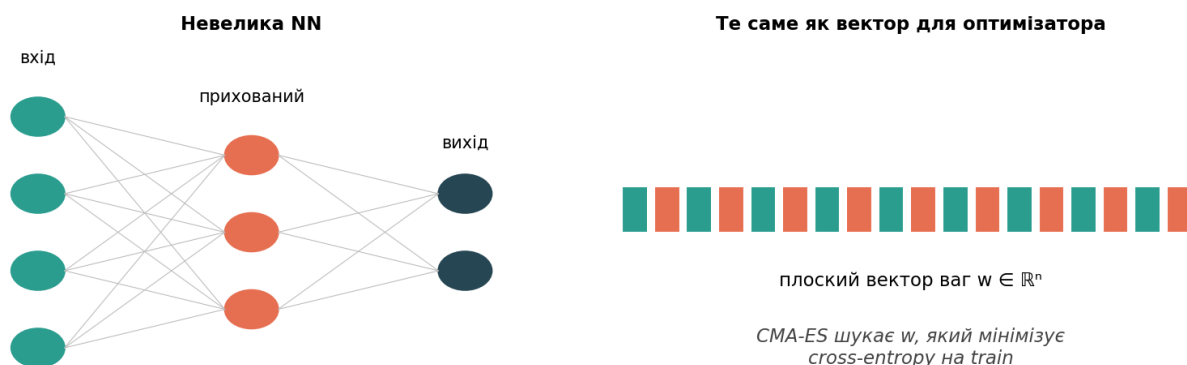


Рис. 5. CMA-NN: ваги — це просто вектор для оптимізатора.

Два режими

- `cma_classic` — оптимізує класичним CMA-ES (пакет `cma`).
- `cma_mixture` — оптимізує розширеним CMA-ES зі сумішами (моя реалізація за Літвінчук).

Особливості

Для швидкості при великих ознаках вмикається PCA (до 20 компонент) і субсемпсування train-вибірки (до 3000 рядків). Це дозволяє CMA-ES не зависати на PhiUSIL з його 235 тисячами рядків і 50 ознаками.

7. Підбір гіперпараметрів (tuned_*)

Для кожного класифікатора додатково є варіант tuned_xxx — це коли CMA-ES використовується не для навчання моделі, а для підбору її гіперпараметрів.

Що шукається для кожної моделі

```
tuned_logreg → C
tuned_svm    → C, gamma
tuned_knn    → n_neighbors
tuned_mlp    → hidden_size, alpha, learning_rate
tuned_gam    → n_knots, degree, C (= 1/λ)
```

tuned_gam — місток між розділами 1 і 2 диплома

Це найцікавіша модель у списку. У розділі 1 диплома описано параметри GAM (k , λ , тип сплайну). У розділі 2 — алгоритм CMA-ES. Модель tuned_gam буквально показує, як алгоритм із розділу 2 шукає оптимальні значення параметрів із розділу 1.

На захисті можна окремо підкреслити цей місток — він підтверджує, що обидва теоретичні розділи з'єднані у практичній реалізації.

8. Як вимірюємо якість

Train / Test split

Спочатку дані ділимо 80/20 зі стратифікацією (зберігаємо пропорцію класів). На train — навчаємо, на test — фінально оцінюємо. test_* метрики у таблиці — це саме ця оцінка.

K-fold cross-validation

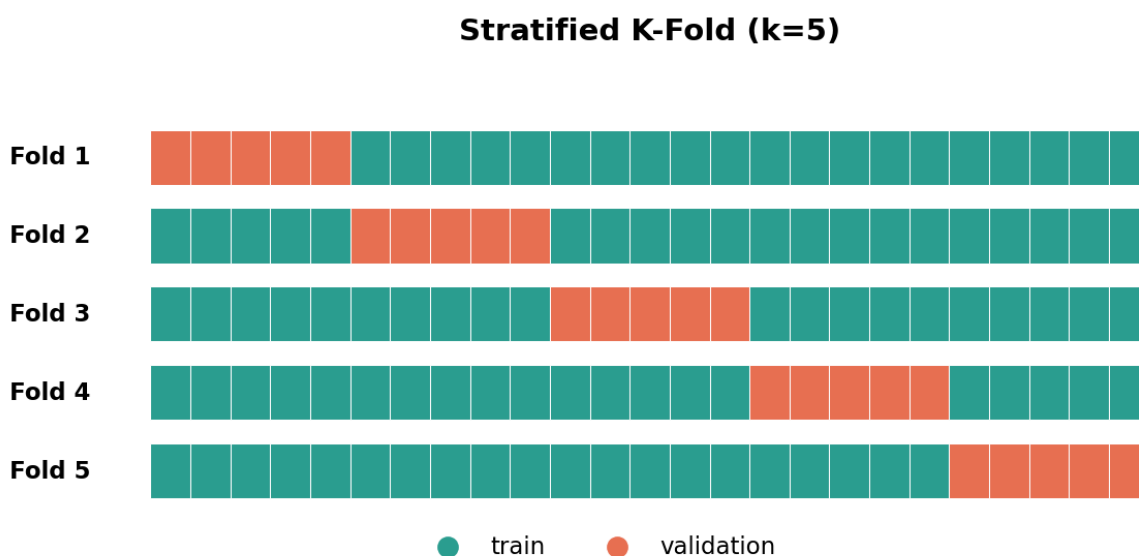


Рис. 6. Stratified K-Fold з k=5.

На тренувальній частині додатково проводимо k-fold CV: train ділиться на k частин (k=5 за замовчуванням). По черзі кожна частина стає валідаційною, інші — тренувальними. Це дає 5 значень кожної метрики — їх середнє (cv_*_mean) і відхилення (cv_*_std).

Великий cv_*_std означає, що модель нестабільна — на різних розбиттях показує дуже різні результати.

Три метрики

- **Ассурасу** — частка правильних прогнозів. Просто, але обманює на незбалансованих класах (Credit Default — типовий приклад: модель, яка завжди передбачає 0, дасть 78% ассурасу і 0% F1).
- **F1-score** — гармонічне середнє precision і recall. Для бінарних задач — стандартний індикатор. Для мультикласу беремо weighted average по класах.
- **ROC-AUC** — площа під ROC-кривою. Показує, наскільки добре модель РАНЖИРУЄ об'єкти за ймовірністю (а не лише класифікує). Для мультикласу — One-vs-Rest weighted average.

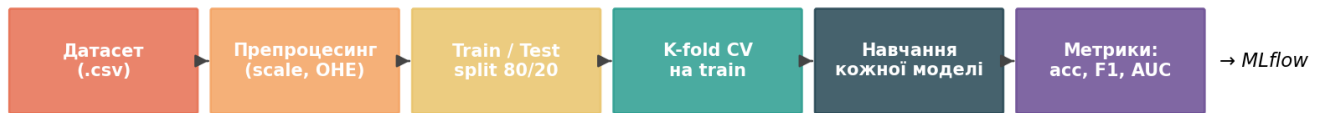
Pipeline: один прогон експерименту

Рис. 7. Загальна послідовність одного експерименту.

9. MLflow — журнал експериментів

MLflow автоматично записує кожен запуск: які параметри, які метрики, скільки часу зайняло. Це потрібно щоб (а) відтворити результати, (б) показати куратору повну історію експериментів.

Що записується

- Параметри: датасет, модель, кількість фолдів, seed, ліміт вибірки і т.д.
- Метрики: test_accuracy, test_f1, test_auc, cv_f1_mean, cv_f1_std, fit_time_s.
- Для tuned_* — додатково знайдені оптимальні гіперпараметри.

Як подивитися

Подвійний клік по run_mlflow.bat, або в терміналі:

```
mlflow ui
```

Браузер відкриє <http://localhost:5000> — там можна порівнювати запуски, фільтрувати, сортувати, експортувати.

10. Як запустити

Найпростіший спосіб — лаунчери

У корені проекту лежать два .bat-файли. Подвійний клік:

- run_app.bat — Streamlit-дашборд на <http://localhost:8501>
- run_mlflow.bat — MLflow UI на <http://localhost:5000>

Через термінал (CLI)

```
.venv\Scripts\activate

# швидкий прогон, без MLflow і tuning
python scripts/run_experiment.py \
  --dataset phiusiil --sample 5000 \
  --folds 3 --no-mlflow --no-tuning

# повний прогон з усіма 12 моделями
python scripts/run_experiment.py \
  --dataset steel_plate --folds 5

# тільки конкретні моделі
python scripts/run_experiment.py \
  --dataset loan_approval --sample 8000 \
  --models gam,tuned_gam,cma_mixture
```

Корисні опції CLI

```
--dataset  phiusiil | steel_plate | loan_approval
--models   all або csv: logreg,svm,gam,...
--folds    кількість фолдів (3-10)
--sample N  підсемпл (PhiUSIIL обов'язково)
--cma-iter N ітерацій CMA-ES (15-100)
--no-mlflow не логувати в MLflow
--no-tuning без tuned_* моделей
```


11. Як читати результати у дашборді

Що ти бачиш

- 5 карток зверху: датасет, train/test розмір, кількість ознак і класів.
- Велика таблиця з усіма моделями. У колонках Accuracy / F1 / AUC — кольорові progress-бари. Шкала zoom'ована по факту значень — тобто бар "наповнений" означає не 1.0, а максимум серед поточних результатів.
- Клік по заголовку колонки сортує таблицю.
- Кнопка "Завантажити CSV" — для звіту в дипломі.
- Гістограма "Час навчання" — порівняння швидкості моделей.
- Графік "Збіжність CMA-ES" — як зменшується cross-entropy по ітераціях для cma_classic і cma_mixture.

На що звертати увагу

- test_f1 і cv_f1_mean мають бути близькі. Якщо test значно вище — пощастило з розбиттям. Якщо нижче — модель перенавчилась.
- cv_f1_std — стабільність. <0.02 = стабільно, >0.05 = нестабільно.
- Час навчання cma_mixture зазвичай у 10-50х більший — це ціна за гнучкість оптимізатора без похідних.
- На PhiUSIII майже всі моделі $\approx 100\%$ (датасет легкий) — цей датасет демонструє коректність всіх реалізацій. Цікаві порівняння — на Credit Default і Steel Plate.

12. FAQ для захисту

Чому СМА-NN не завжди перемагає sklearn-моделі?

Бо це інший інструмент. СМА-NN — це оптимізатор БЕЗ ПОХІДНИХ. Він універсальний (можна оптимізувати майже будь-що), але повільніший і менш точний, ніж градієнтні методи на типових функціях. На задачах із мультимодальним ландшафтом, де градієнтні методи застрягають, СМА-NN може виграти — це і є основна цінність методу зі сумішами.

У чому новизна порівняно з дисертацією Літвінчук?

- Чисте ЕМ-оновлення коваріацій (як у дисертації) на унімодальних задачах призводить до колапсу дисперсії. Я додав параметр `cov_lr` — момент-усереднення коваріації між ітераціями. Це невелика, але необхідна модифікація для практичного застосування.
- У дисертації — оптимізація на тестових функціях. У моїй роботі — застосування до реальної задачі класифікації на трьох сучасних датасетах.
- Реалізовано міст $\text{GAM} \leftrightarrow \text{CMA-ES}$ (модель `tuned_gam`) — об'єднання обох розділів теоретичної частини диплома.

Чому використано не Kaggle, а UCI варіанти двох датасетів?

Kaggle-варіанти потребують API-токену і реєстрації. UCI — відкриті, скачуються одним рядком. Програма підтримує обидва варіанти: якщо `kaggle.json` є — береться Kaggle (2024), якщо немає — UCI fallback. На функціональність порівняння це не впливає. Мінімум "хоч 1 датасет $\leq$ 2-3 років" закритий PhiUSIIL 2024.

Як було тестовано?

Pytest-набір із 34 тестів покриває: завантаження датасетів, препроцесинг, кожен класифікатор, обидва СМА-ES, ЕМ-крок, збіжність на тестових функціях (сфера, Розенброк, бімодальна), tuning. Запуск — ``pytest -q`` у корені проекту.

Чому Streamlit, а не власний UI?

Streamlit — стандартний фреймворк для ML-демо. Один файл Python = повноцінний веб-дашборд з графіками, фільтрами, експортом. Окрім дашборду, MLflow дає вебінтерфейс журналу експериментів.

Корисні файли і команди

```
Тести:          pytest -q
Запустити дашбورد: run_app.bat
MLflow UI:      run_mlflow.bat
Згенерувати цей PDF: python scripts/generate_guide.py

Git log:        git log --oneline
Поточний стан:  git status
```

Якщо щось перестане працювати — починай з ``pytest -q``. Якщо тести проходять, проблема в інтеграції/налаштуванні, а не в логіці.